

Operating Systems

Chapter 3: Deadlock

Zahra Jafarinassab (z.jafarinassab@gmail.com)

Contents:

- 1- System Model
- 2- Deadlock Characterization
- 3- Resource-Allocation Graph
- 4- Methods for Handling Deadlocks
- 5- Deadlock Prevention
- 6- Deadlock Avoidance
- 7- Safe State
- 8- Avoidance Algorithms
- 9- Banker's Algorithm
- 10- Deadlock Detection

1-System Model

- In a multiprogramming environment, several threads may compete for a finite number of resources. A thread requests resources; if the resources are not available at that time, the thread enters a waiting state. Sometimes, a waiting thread can never again change state, because the resources it has requested are held by other waiting threads. This situation is called a **deadlock**
- Operating systems typically do not provide deadlock-prevention facilities, and it remains the responsibility of programmers to ensure that they design deadlock-free programs
- System consists of resources

- Resource types $R_1, R_2, \dots, R_m$
 - CPU cycles, Files, Memory Space, I/O devices (DVD Driver, Printer, ...)
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **Request.** The thread requests the resource. If the request cannot be granted immediately (for example, if a mutex lock is currently held by another thread), then the requesting thread must wait until it can acquire the resource.
 - **Use.** The thread can operate on the resource (for example, if the resource is a mutex lock, the thread can access its critical section).
 - **Release.** The thread releases the resource.

2-Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously. All four conditions must hold for a deadlock to occur.

- **Mutual exclusion:** only one thread at a time can use a resource
- **Hold and wait:** a thread holding at least one resource is waiting to acquire additional resources held by other threads
- **No preemption:** a resource can be released only voluntarily by the thread holding it, after that thread has completed its task
- **Circular wait:** there exists a set $\{T_0, T_1, \dots, T_n\}$ of waiting threads such that T_0 is waiting for a resource that is held by T_1 , T_1 is waiting for a resource that is held by T_2 , ..., T_{n-1} is waiting for a resource that is held by T_n , and T_n is waiting for a resource that is held by T_0 .

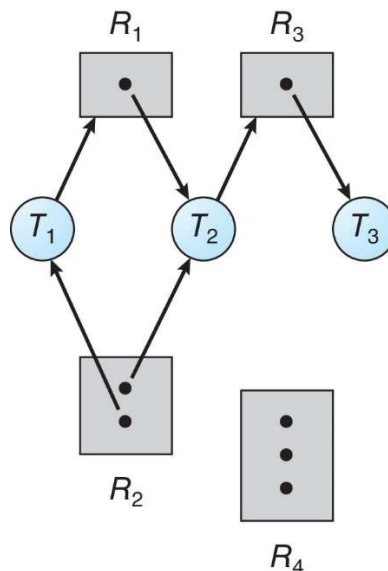
3-Resource-Allocation Graph

Deadlocks can be described more precisely in terms of a directed graph called a **system resource-allocation graph**: A set of vertices V and a set of edges E .

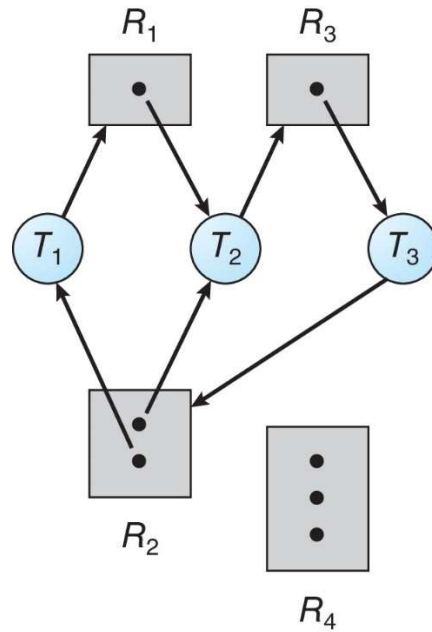
- V is partitioned into two types:
 - $T = \{T_1, T_2, \dots, T_n\}$, the set consisting of all the threads in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $T_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow T_i$

Example:

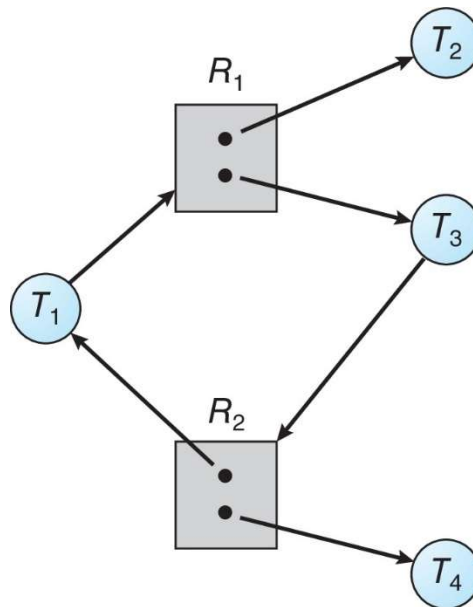
- One instance of R1
- Two instances of R2
- One instance of R3
- Three instances of R4
- T1 holds one instance of R2 and is waiting for an instance of R1
- T2 holds one instance of R1, one instance of R2 & is waiting for an instance of R3
- T3 holds one instance of R3



Resource Allocation Graph with a Deadlock:



Graph with a Cycle But no Deadlock:



- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock
 - if several instances per resource type, possibility of deadlock

4-Methods for Handling Deadlocks

- Ensure that the system will never enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system.

5-Deadlock Prevention

Invalidate one of the four necessary conditions for deadlock:

- Mutual Exclusion – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources
- Hold and wait – must guarantee that whenever a thread requests a resource, it does not hold any other resources
 - Require threads to request and be allocated all its resources before it begins execution or allow thread to request resources only when the thread has none allocated to it.
 - Low resource utilization; starvation possible
- No Preemption:
 - If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
 - Preempted resources are added to the list of resources for which the thread is waiting
 - Thread will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

- Circular Wait:
 - Impose a total ordering of all resource types, and require that each thread requests resources in an increasing order of enumeration

6-Deadlock Avoidance

Requires that the system has some additional *a priori* information available

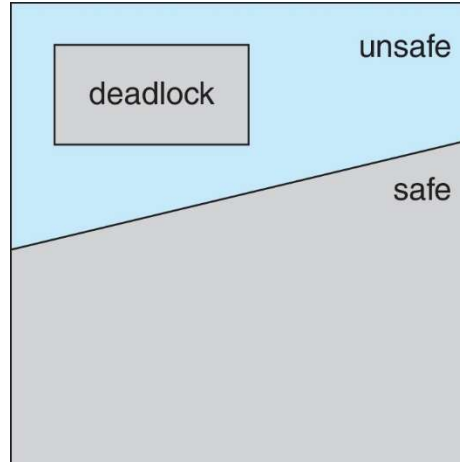
- Simplest and most useful model requires that each thread declare the **maximum number** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

7-Safe State

- When a thread requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- System is in **safe state** if there exists a sequence $\langle T_1, T_2, \dots, T_n \rangle$ of ALL the threads in the systems such that for each T_i , the resources that T_i can still request can be satisfied by currently available resources + resources held by all the T_j , with $j < i$
- That is:
 - If T_i resource needs are not immediately available, then T_i can wait until all T_j have finished
 - When T_j is finished, T_i can obtain needed resources, execute, return allocated resources, and terminate
 - When T_i terminates, T_{i+1} can obtain its needed resources, and so on

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State:



8-Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
 - ▶ Suppose that thread T_i requests a resource R_j
 - ▶ The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph
- Multiple instances of a resource type
 - Use the Banker's Algorithm

9-Banker's Algorithm

- Multiple instances of resources
- Each thread must a priori claim maximum use
- When a thread requests a resource, it may have to wait
- When a thread gets all its resources it must return them in a finite amount of time

Data Structures for the Banker's Algorithm:

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process T_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then T_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then T_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Safety Algorithm:

- 1) Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize:

Work = Available

Finish $[i] = false$ for $i = 0, 1, \dots, n-1$

- 2) Find an **i** such that both:

(a) **Finish $[i] = false$**

(b) **$Need_i \leq Work$**

If no such **i** exists, go to step 4

3) $Work = Work + Allocation_i$

$Finish[i] = true$

go to step 2

4) If $Finish[i] == true$ for all i , then the system is in a safe state

Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process T_i . If $Request_i[j] = k$ then process T_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise T_i must wait, since resources are not available
3. Pretend to allocate requested resources to T_i by modifying the state as follows:

$Available = Available - Request_i;$

$Allocation_i = Allocation_i + Request_i;$

$Need_i = Need_i - Request_i;$

- ▶ If safe $\Rightarrow$ the resources are allocated to T_i
- ▶ If unsafe $\Rightarrow T_i$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm:

- 5 threads T_0 through T_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

- The content of the matrix **Need** is defined to be **Max - Allocation**

	<u>Need</u>
	<i>A B C</i>
T_0	7 4 3
T_1	1 2 2
T_2	6 0 0
T_3	0 1 1
T_4	4 3 1

- The system is in a safe state since the sequence $\langle T_1, T_3, T_4, T_2, T_0 \rangle$ satisfies safety criteria

Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow \text{true}$)

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
T_0	0 1 0	7 4 3	2 3 0
T_1	3 0 2	0 2 0	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle T_1, T_3, T_4, T_0, T_2 \rangle$ satisfies safety requirement
- Can request for (3,3,0) by T_4 be granted?
- Can request for (0,2,0) by T_0 be granted?

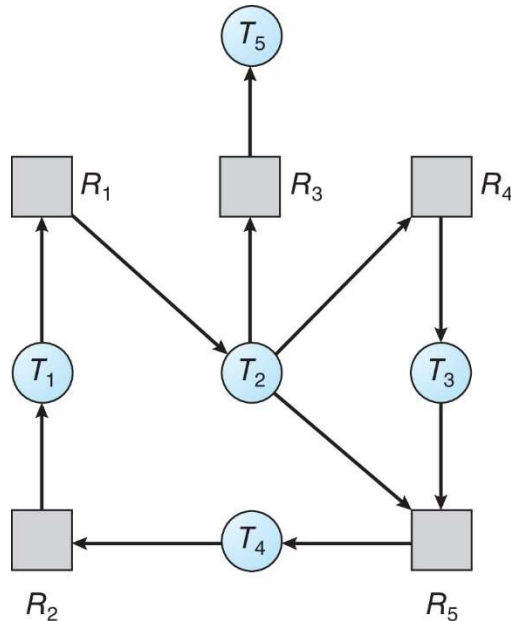
10- Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

Single Instance of Each Resource Type:

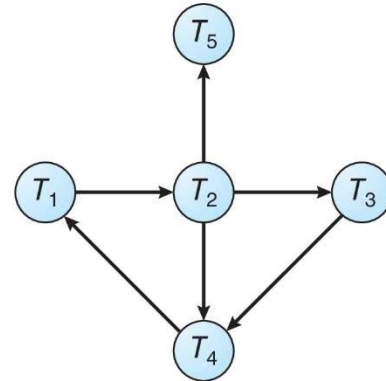
- Maintain **wait-for** graph
 - Nodes are threads
 - $T_i \rightarrow T_j$ if T_i is waiting for T_j
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

Resource-Allocation Graph and Wait-for Graph:



(a)

Resource-Allocation Graph



(b)

Corresponding wait-for graph

Several Instances of a Resource Type:

- **Available:** A vector of length m indicates the number of available resources of each type
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each thread.
- **Request:** An $n \times m$ matrix indicates the current request of each thread. If $Request[i][j] = k$, then thread T_i is requesting k more instances of resource type R_j .

Detection Algorithm:

1. Let **Work** and **Finish** be vectors of length m and n , respectively Initialize:

A) **Work** = **Available**

B) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then

Finish[i] = **false**; otherwise, **Finish**[i] = **true**

2. Find an index i such that both:

A) $Finish[i] == false$

B) $Request_i \leq Work$

If no such i exists, go to step 4

3. $Work = Work + Allocation_i$

$Finish[i] = true$

go to step 2

4. If $Finish[i] == false$, for some i , $1 \leq i \leq n$, then the system is in deadlock state.
Moreover, if $Finish[i] == false$, then T_i is deadlocked

Example:

Five threads T_0 through T_4 ; three resource types
A (7 instances), B (2 instances), and C (6 instances)

▪ Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
T_0	0 1 0	0 0 0	0 0 0
T_1	2 0 0	2 0 2	
T_2	3 0 3	0 0 0	
T_3	2 1 1	1 0 0	
T_4	0 0 2	0 0 2	

- Sequence $\langle T_0, T_2, T_3, T_1, T_4 \rangle$ will result in $Finish[i] = true$ for all i
- T_2 requests an additional instance of type C

Request

A B C

T_0 0 0 0

T_1 2 0 2

T_2 0 0 1

T_3 1 0 0

T_4 0 0 2

- State of system?
 - Can reclaim resources held by thread T_0 , but insufficient resources to fulfill other processes; requests
 - Deadlock exists, consisting of processes T_1 , T_2 , T_3 , and T_4

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - ▶ one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked threads “caused” the deadlock.

Recovery from Deadlock: Process Termination

- Abort all deadlocked threads
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?

- 1) Priority of the thread
- 2) How long has the thread computed, and how much longer to completion
- 3) Resources that the thread has used
- 4) Resources that the thread needs to complete
- 5) How many threads will need to be terminated
- 6) Is the thread interactive or batch?

Recovery from Deadlock: Resource Preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart the thread for that state
- **Starvation** – same thread may always be picked as victim, include number of rollbacks in cost factor